Supplementary Material for "GVS5H: Zero-Shot Self-Orchestration With Ledger-Based Control Improves Coding in Language Models"

Anonymous Authors

I. Two Corrections to the LiveCodeBench Evaluator

TABLE S1
Every Reported Arm as the Stock Evaluator Graded It, and After Each Correction

	Single call				With manager			
Model	**As graded**	**Δ harness**	**Δ keys**	**Re-ported**	**As graded**	**Δ harness**	**Δ keys**	**Re-ported**
Pinned backends, five passes								
Claude Fable 5	87.4	+0.0	+3.0	**90.4**	—	—	—	—
GPT-5.6-Terra	69.8	+8.4	+2.6	**80.8**	72.6	+12.6	+2.8	**88.0**
GPT-5.6-Luna	60.4	+8.0	+2.0	**70.4**	70.4	+8.4	+2.4	**81.2**
Qwen3.8-27B	62.4	+2.6	+1.8	**66.8**	84.4	+5.4	+2.6	**92.4**
Qwen3.8-Flash-Next	78.8	+2.4	+3.0	**84.2**	87.4	+2.6	+3.0	**93.0**
DeepSeek-V4.1-Flash	84.2	+1.2	+3.0	**88.4**	87.6	+1.4	+3.0	**92.0**
OpenRouter-served, one pass								
Opus-5	85.0	+0.0	+3.0	**88.0**	91.0	+1.0	+3.0	**95.0**
Kimi-K3	82.0	+1.0	+3.0	**86.0**	82.0	+1.0	+3.0	**86.0**
MiniMax-M3	60.0	+0.0	+2.0	**62.0**	66.0	+0.0	+3.0	**69.0**
Qwen3.6-35B	25.0	+0.0	+1.0	**26.0**	43.0	+0.0	+1.0	**44.0**

(Section I). **As graded** is LiveCodeBench's own verdict on the stored generations. **Harness** is the move contributed by running each submission as a subprocess, which is what repairs the `sys.stdin` mock and `sys.stdout.buffer`; **keys** is the move contributed by judging the four problems that accept more than one answer by the contest's rule. **Reported** is the number used everywhere else in this paper. Upper block: five-pass means at a 128k cap, reasoning on (Table I of the main paper). Lower block: the 128k reasoning-on, one-pass OpenRouter arms (Table IV of the main paper); Qwen3.5-9B is unusable in this condition and is omitted. Across the 48 arm-passes of the two remaining OpenRouter conditions the harness correction moves six by one point and the key correction moves each arm by 0 to 3 points. Qwen3.8-27B's single call is the one row where **as graded** is not the stock evaluator on the code being reported: that arm is cap-matched by replaying its 250k generations at 128k (Appendix A of the main paper) and the truncated solutions were never put through the stock evaluator, so the column is the 250k grading and the two Δ columns carry the cap match as well as the corrections. Against that baseline the row gains 35 problem-passes and loses 13, eight of them generations whose 128k prefix contains no code at all.

While inspecting transcripts for Section IV of the main paper we found two defects in the grading path: one in the LiveCodeBench harness, one in its answer keys. Every number in Section III of the main paper is reported after fixing both; no model was re-run. Table S1 gives every reported arm as the stock evaluator graded it and after each correction. Neither correction is a constant offset, and the larger of the two is the answer keys for some models and the harness for others.

A. The `sys.stdin` Mock

The evaluator does not run a candidate as a subprocess. Instead, it executes it in-process with `sys.stdin` replaced by a mock. That mock's binary view implemented `readline()` as `inputs.split(b"\n")[0]`, which is a stateless expression that returns the first line on every call. A program reading multi-line input through `sys.stdin.buffer.readline()` therefore read line 1 repeatedly and failed however correct it was. The mock's text-mode `sys.stdin.readline` was implemented as a proper iterator and `buffer.read()` returns the whole payload, so neither was affected, which is why the common `sys.stdin.buffer.read().split()` idiom never exposed it. We replaced the mock's binary view with one backed by a `BytesIO` so that reads advance a position, and re-scored every stored generation.

We report it explicitly for two reasons. First, it interacts badly with the v2 verifier: that step runs the candidate as a real subprocess (step 5 of Section II of the main paper), where `buffer.readline()` works correctly, so the manager was told its solution passed the public samples for programs the grader then marked wrong - the loop's one external signal certifying a wrong answer. Second, the exposure is uneven across models rather than a constant offset: 371 of the 5,425 pinned-backend outputs containing code use the idiom, and 86% of those were scored wrong against 18% for the outputs that do not. Fable 5 never uses it and its scores are unchanged to the decimal. The OpenRouter-served set reaches for it once in 4,764 generations containing code, and then through an alias - Kimi-K3's single call on `abc396_g` binds `sys.stdin.buffer` to a name and calls `readline()` on that. Every OpenRouter-served number is re-scored all the same, and the repaired mock moves one cell: Kimi-K3's 128k reasoning-on single arm, from 82 to 83 (Table S1, where that cell reaches its reported 86 through the answer-key correction below). Because the idiom is a matter of coding style, a harness bug of this shape is not a wash across a leaderboard - it penalizes the models whose style trips it, and its Δ harness column ranges from +0.0 for Fable 5, which never writes it, to +12.6 for GPT-5.6-Terra's manager arm.

B. Answer Keys That Accept One of Many Valid Answers

LiveCodeBench grades by exact string match against a single stored output. Four LCB-100 problems accept more than one: `abc396_e`, `arc190_a` and `arc195_c` ask for any construction that attains the optimum, and `abc385_f` asks for a real number, which its contest accepts within a relative 10^{-6}. We judge those four by the rule their contest used. Each of the three constructive problems gets a checker that parses the submitted answer, tests it against the problem's own constraints, and requires it to attain the objective value of the stored reference: for `abc396_e`, that the sequence is non-negative, satisfies every XOR constraint and sums to what the reference sums to; for `arc190_a`, that the chosen operations cover $1..N$ and number what the reference says is optimal; for `arc195_c`, that the Yes/No verdict matches the reference and that any placement offered uses the requested counts of each colour, on distinct cells, with every consecutive pair a step of the kind its colour allows. `abc385_f` is compared token by token at 10^{-6} relative tolerance. This class of defect is one LiveCodeBench documents upstream - its errata list eleven earlier problems whose keys accept multiple answers, none of them in LCB-100 - and it is a property of the benchmark, not of any model, so it applies to every arm of both sets and moves 104 of their 118 arm-passes. Re-judging runs each submission as a real subprocess rather than in-process, which is also what repairs the mock above and makes `sys.stdout.buffer` write as it does on the contest judge.

Both corrections are conservative in the same two ways. They re-run only the submissions the stock evaluator marked wrong, so neither can turn a pass into a failure and the corrections move every arm of Table S1 upward; and among those they re-run only the ones with a known reason to be misjudged - a failure on one of the four problems above, or code that touches `sys.stdin.buffer` or `sys.stdout.buffer`. A submission that the in-process evaluator failed for some other reason keeps that verdict. Re-executing every stored generation instead, passing and failing alike, finds 25 further submissions across 11,800 records that pass every hidden test as a subprocess; they are spread over 22 arms, at most three to an arm, and we leave them as the stock evaluator scored them.

II. Experimental Conditions

The pinned-backend set runs at one setting: 128k output cap, reasoning on, $\times 5$ passes, with the five models in both arms and Fable 5 single-only. The OpenRouter-served set runs at three: 128k reasoning-on $\times 1$; 16k reasoning-off $\times 5$, whose repeated passes give run-to-run variance and paired tests; and 128k reasoning-off $\times 1$, the same without the small cap, isolating it as a confound.

III. Cost per Arm

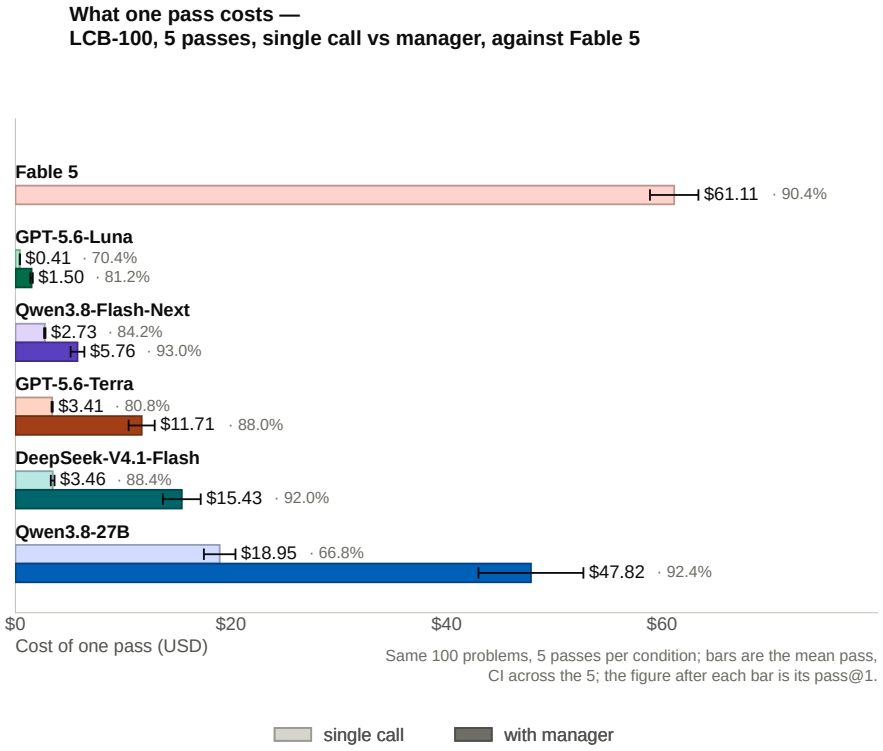


Fig. S1. The scaffold's bill. Cost of one pass over the same 100 problems; Table II of the main paper is the arithmetic behind every bar. No cached-input discount is taken, and Qwen3.8-27B is priced at OpenRouter market rates. The top bar of each pair is the single call, light; the manager is under it, dark; all are at a 128k output cap, and colour is the model. Fable 5 is first; the models below it run top to bottom from the cheapest single call to the dearest. Table II carries the tests. Unbracketed comparisons: every manager − single increase is significant, and GPT-5.6-Luna's manager undercuts GPT-5.6-Terra's single call by $1.91. At 1.28x the assumed Qwen rate ($0.273/$3.26 per MTok, inside the spread across hosted providers) the manager's cost advantage over Fable 5 disappears entirely - a larger uncertainty than any p-value here.

IV. Accuracy by Model Scale

Figs. S2 and S3 plot the one-pass OpenRouter-served results of Fig. 4 of the main paper against model parameter count; the five-pass condition is Fig. 5 of the main paper.

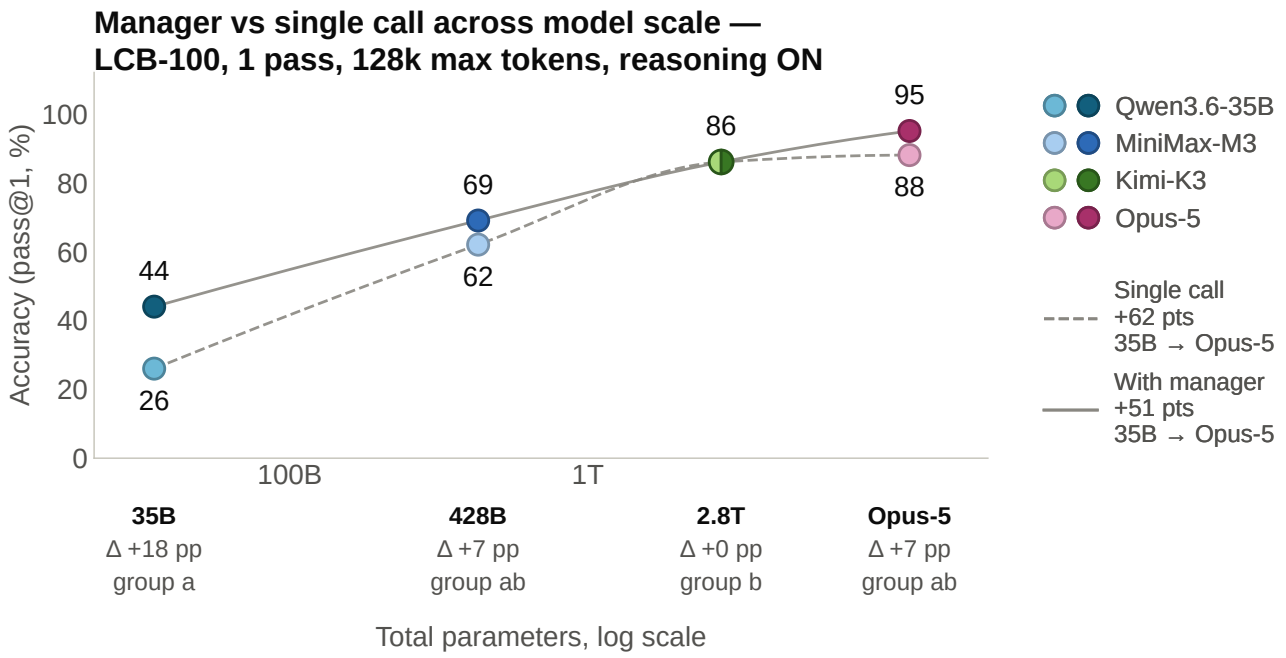


Fig. S2. Accuracy by model scale, reasoning on, 128k, one pass. The same numbers as Fig. 4(a), against parameter count. Dashed = single call, solid = with manager; the block under each tick gives that model's Δ and Tukey group, and the legend each arm's gain from the smallest model to the largest.

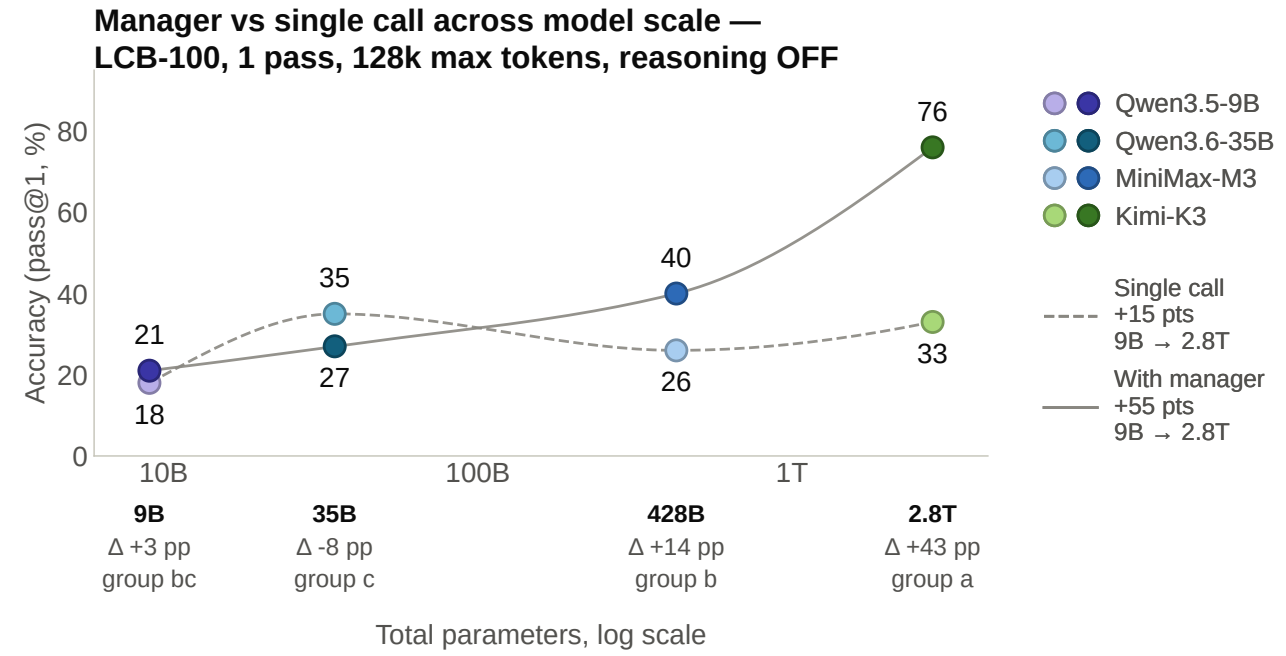


Fig. S3. Accuracy by model scale, reasoning off, 128k, one-pass control. The same numbers as Fig. 4(c), against parameter count. Dashed = single call, solid = with manager; the block under each tick gives that model's Δ and Tukey group, and the legend each arm's gain from the smallest model to the largest.